

课程笔记

阿里云亮出 Agent 基础设施全景图： ANOLISA 要做每一个 Agent 的运行底座

五道口纳什 & Codex

2026 年 6 月 6 日



视频作者/频道：龙蜥社区 __ 小龙

发布日期：2026-05-29

视频时长：00:20:18

视 频 链 接：https://www.bilibili.com/video/BV1o2Vh6EEAe/?spm_id_from=333.337.top_right_bar_window_history.content.click&vd_source=6457e444c35aa58365caef6c074dd705

目录

1 Agent 成为操作系统的新用户	3
1.1 企业应用的 Agent 化速度	3
1.2 人类用户假设的松动	3
1.3 ANOLISA 的任务目标	4
1.4 本章小结	5
2 真实案例：Python 服务部署暴露环境探索成本	5
2.1 5 分钟任务与 14 轮对话	5
2.2 环境探索怎样吞掉 token	6
2.3 从人机交互循环到运行时契约	7
2.4 本章小结	8
3 ANOLISA 的定位与三层架构	8
3.1 它服务谁：Agent 与托管平台	8
3.2 叠加传统 Linux 的升级路线	9
3.3 系统优化层、运行时层、封装交互层	10
3.4 本章小结	11
4 从四项优势到 Token 优化：少想、少装、少传	11
4.1 四项优势的总览	11
4.2 应用层优化与 OS 层优化的叠加	12
4.3 少想：OS Skill 与环境地图	13
4.4 少装：Skill 文件系统与上下文裁剪	13
4.5 少传：输入输出精简	14
4.6 Dashboard 证明节省发生在哪里	14
4.7 本章小结	16
5 Copilot Shell：把 OS 能力变成意图式入口	16
5.1 从语法式 CLI 到意图式交互	16
5.2 统一 gateway 如何暴露 OS 能力	17
5.3 Demo 1：一句自然语言拉起服务	18
5.4 Demo 2：通过 Agent 接口完成云盘扩容	19
5.5 本章小结	20
6 高密 Agent 运行时：小规格实例上的并发、内存与调度	20
6.1 每名员工 10 到 100 个 Agent 的负载假设	20
6.2 小规格实例上的三类压力	20
6.3 高密部署与互不干扰	21
6.4 ACS 场景、突发算力与长稳运行	22
6.5 本章小结	23

7 Agent 安全：执行前、执行中、OS 层兜底	23
7.1 Agent 安全为什么决定自主执行边界	23
7.2 确定性防护模型的局限	24
7.3 三层防御架构	25
7.3.1 第一层：执行前拦截	25
7.3.2 第二层：执行中监控	26
7.3.3 第三层：OS 层隔离与安全基线	26
7.4 本章小结	27
8 总结与延伸	27
8.1 讲者的收束：开放底座才有生态价值	27
8.2 GitHub 开源与龙蜥 Skill Hub	27
8.3 UNIX 例子与接口思想	29
8.4 全篇综合：从成本痛点到生态承载	29
8.5 读者检查清单：评估自己的 Agent 工作负载	30
8.6 本章小结	31

1 Agent 成为操作系统的新用户

1.1 企业应用的 Agent 化速度

这场分享的入口很直接：周旭来自阿里云基础软件团队，负责 Agentic OS 相关产品，他把 ANOLISA 放在“Agent 系统管家”的位置上，目标是打造更高效、更安全的 Agent Native 环境。开场给出的背景数字是，到年底约 40% 的企业应用会内置 AI Agent。这个数字在本章里只承担一个作用：说明企业应用的默认交互对象正在变化，操作系统面对的“用户”范围也随之扩大。



视频时间：00:00:07-00:00:13

图 1：开场标题页展示了 ANOLISA 的主题定位：Agent 系统管家，目标是打造更高效、更安全的 Agent Native 环境。

核心定义：AI Agent 是新的 OS 用户主体

本章中的 AI Agent 指能够接收目标、调用工具、读取环境、修改系统状态并持续推进任务的软件执行主体。它在操作系统面前具有“用户”的行为特征：会发起请求、读取反馈、选择下一步动作，也会消耗 CPU、内存、文件、网络、权限和日志等系统资源。

把 AI Agent 看成“用户主体”，含义比“多了一个自动脚本”更重。自动脚本通常按固定流程执行，失败后等待人处理；Agent 会根据中间结果继续观察、推理和试错。对于操作系统来说，这意味着一次任务可能变成多轮命令、多次目录扫描、多段日志读取、多次权限尝试和多次服务重启。模型变聪明只能提升决策能力，系统交互成本依然会直接进入账单。

1.2 人类用户假设的松动

过去几十年的 Linux 设计默认主要用户是人。这个“人类用户假设”体现在很多细节里：人看屏幕，人敲键盘，人能等几秒，人能读一大段日志，人会凭经验跳过无关步骤，人会在危险命令前停一下。命令行、日志、错误码、权限提示、目录结构和包管理工具，很多地方都在服务这种人机循环。

AI Agent 进入服务器侧任务后，这个假设开始松动。Agent 不看屏幕，也不敲键盘；它通过文本、API、工具调用和系统返回值理解世界。更关键的是，它可能在服务器上 7×24 小时持续运行，像一支不停接单自动运维队伍。此时，操作系统给出的每一段模糊输出、每一次迟缓响应、每一条需要人工直觉解释的日志，都会被放大成推理成本和调用轮次。

这个变化可以用一个极简公式压缩：

$$p_{\text{enterprise}} = 40\%, \quad H_{7 \times 24} = 7 \times 24 = 168 \text{ h/week}$$

符号说明：

- $p_{\text{enterprise}}$ ：开场提到的企业应用内置 AI Agent 的背景比例。
- 40%：用于说明 Agent 普及速度的比例，不能直接推导市场规模。
- $H_{7 \times 24}$ ：Agent 服务器侧持续任务负载的周运行时长。
- 168 h/week：每周 168 小时，表示任务主体从间歇人工操作扩展到连续运行。

一个更直观的类比

传统 Linux 像一栋给人类员工使用的办公楼：门牌、楼层、开关和告示都默认有人会看、会问、会判断。Agent 进入后，办公楼里多了一批自动机器人，它们昼夜巡回、按指令开门、读告示、搬材料。原来的楼没有塌，但告示写法、通道标识、权限闸门和故障反馈都需要适配新的使用者。

1.3 ANOLISA 的任务目标

ANOLISA（Agent 系统管家）在本章中定义为：阿里云面向 AI Agent 推出的系统级管家产品，使命是为 Agent 构建高效、安全的 Agent Native 环境。这里的 Agent Native 环境，指运行环境把 Agent 的交互方式、持续运行特征、系统资源消耗和安全边界作为一等设计对象，避免把 Agent 简化为传统人类用户背后的一个普通进程。

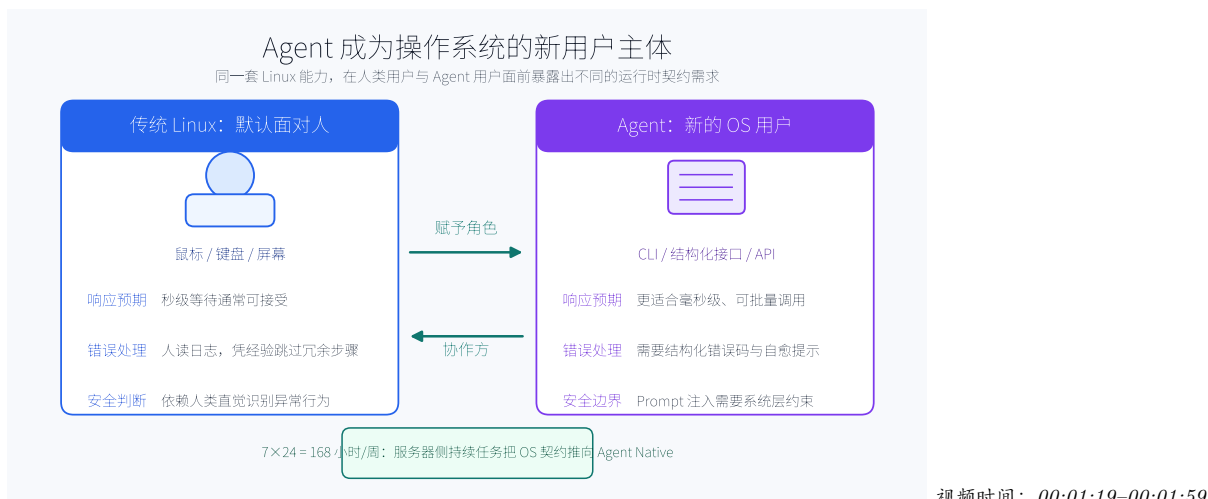


图 2：教学示意图重构了讲者在该时段说明的核心差异：传统 Linux 以人类交互为默认对象，而 Agent 需要面向 7×24 服务器任务的结构化接口、错误处理和安全边界。

“系统管家”这个词很关键。管家不直接替业务写应用，也不替模型做推理；它负责让运行环境更可理解、更可控、更省成本。Agent 要部署服务、读日志、找目录、处理权限、识别冲突，系统管家就应该把这些系统侧知识整理成 Agent 更容易消费的形式，并在安全和资源边界上给出稳定约束。

常见误解：模型更强就能自动消除系统成本

模型能力提升会降低一部分推理错误，却无法自动消除环境探索。只要操作系统仍用面向人类的方式返回模糊日志、分散状态和人工化错误提示，Agent 就会继续用多轮调用去补齐上下文。成本问题因此落到了模型、工具和 OS 共同组成的执行链上。

本章的结论到这里已经足够清楚：Agent 成为新的 OS 用户主体后，操作系统需要重新理解“谁在用系统、怎样用系统、失败时怎样恢复、风险怎样被限制”。下一章会把这个抽象变化落到一个具体案例上：一个熟练工程师 5 分钟能完成的 Python 服务部署，为什么会让 Agent 花掉 14 轮对话。

1.4 本章小结

本章建立了三个基础定义。第一，AI Agent 是新的 OS 用户主体，它会持续调用工具、读取反馈并改变系统状态。第二，Agent Native 环境把 Agent 的运行方式放进操作系统设计目标中，尤其关注高频交互、长期运行、资源消耗和安全边界。第三，ANOLISA 是面向 Agent 的系统管家，承担的是运行环境层面的整理、约束和加速任务。理解这三点后，Python 服务部署案例就会呈现为 Agent 与传统 OS 交互结构失配的一次具体暴露。

2 真实案例：Python 服务部署暴露环境探索成本

2.1 5 分钟任务与 14 轮对话

讲者在介绍产品之前先给出一个真实任务：让 AI Agent 在既有环境中部署一个 Python 服务。对于熟练工程师，这类任务通常在 5 分钟之内完成；同一任务交给 Agent 后，最终用了 14 轮对话。更值得注意的数字是，前 13 轮主要花在目录结构梳理、权限问题解决和冲突处理上，第 14 轮才把 Python 服务完整部署下来。



视频时间：

00:02:06-00:03:10

图 3: Python 服务部署案例把成本问题具体化：前 13 轮对话主要用于目录、权限、依赖和端口等环境探索，第 14 轮才开始真正执行部署任务，约 80% 的 Token 消耗在探索与试错上。

这个案例把“越跑越贵”拆成了可观察链条：任务目标很小，执行路径很长，长出来的部分集中

在环境探索。熟练工程师看到目录、日志和报错后，会用经验快速判断哪些信息有用；Agent 需要把这些经验转换成一次次工具调用。每一轮调用又会带来新的输入 token、输出 token、日志 token 和推理 token。

先看轮次占比：

$$R_{\text{agent}} = 14, \quad R_{\text{setup}} = 13, \quad q_R = \frac{R_{\text{setup}}}{R_{\text{agent}}} = \frac{13}{14} \approx 92.9\%$$

符号说明：

- R_{agent} ：Agent 完成 Python 服务部署所用的总对话轮数。
- R_{setup} ：用于目录结构、权限和冲突处理的设置与探索轮数。
- q_R ：探索轮次占总轮次的比例。
- 92.9%：本案例中环境探索在轮次层面的占比。

核心定义：环境探索

环境探索指 Agent 为搞清当前系统状态而进行的目录查看、依赖确认、权限试探、日志读取、端口检查、冲突定位和失败重试。它有时是必要步骤，问题在于传统 OS 把这些信息分散给人读，Agent 会用大量轮次把它们重新拼成可行动的上下文。

2.2 环境探索怎样吞掉 token

Token 浪费在本章中有一个精确定义：完成任务目标之外，为理解环境、修复可预期错误、重复传输冗余上下文和试错恢复所消耗的 token。它并非所有额外 token 的总称；如果某段 token 直接改变部署结果，比如安装缺失依赖或生成必要配置，它属于任务执行成本。若 token 主要用来反复确认“当前环境是什么、错误在哪里、权限是否够、冲突来自哪里”，它就属于环境探索成本。

把 token 成本写成一个教学模型，可以先拆成输入和输出两部分：

$$C_{\text{token}} = p_{\text{in}}T_{\text{in}} + p_{\text{out}}T_{\text{out}}$$

符号说明：

- C_{token} ：一次 Agent 任务对应的 token 成本。
- p_{in} ：输入 token 的单位价格或权重。
- T_{in} ：输入侧 token 数，包括用户目标、历史上下文、工具结果和日志片段。
- p_{out} ：输出 token 的单位价格或权重。
- T_{out} ：输出侧 token 数，包括 Agent 的推理文本、工具调用参数和解释性反馈。

讲者给出的关键比例是，大约八成 token 消耗在环境探索与试错上。用公式表达就是：

$$w_{\text{env}} = \frac{T_{\text{env}}}{T_{\text{total}}} \approx 0.8$$

符号说明：

- w_{env} ：环境探索 token 在总 token 中的占比。

- T_{env} : 用于环境探索与试错的 token 数。
- T_{total} : 任务总 token 数。
- 0.8: 讲者给出的约八成比例。

从内部数据看, Agent 比人类多消耗的调用轮次大约在 3 到 5 倍水平:

$$m_{\text{round}} = \frac{R_{\text{agent}}}{R_{\text{human}}} = 3-5$$

符号说明:

- m_{round} : Agent 相对人类的调用轮次倍数。
- R_{agent} : Agent 完成同类任务的调用轮次。
- R_{human} : 人类完成同类任务所需的可比交互轮次。
- 3-5: 讲者提到的内部数据区间。

边界提醒: 3 到 5 倍只是观察区间

这个倍数来自讲者描述的内部观察, 适合说明企业 Agent 任务中的环境探索问题。它不能外推成所有模型、所有系统、所有任务都会稳定多出 3 到 5 倍轮次。任务越依赖本地环境、权限状态、隐式配置和日志判断, 这个倍数通常越容易升高。

2.3 从人机交互循环到运行时契约

接下来, 讲者把部署案例推进到更底层的人机交互循环。人类用户和 OS 交互时, 会经历感知、理解、决策、执行四个环节。人能接受秒级响应, 能读懂长日志, 能凭直觉跳过冗余步骤, 也能在可疑输入或异常行为面前做安全判断。Agent 的交互节奏更紧: 它希望系统更快返回, 错误更结构化, 风险信号更明确。

核心定义: 运行时契约

运行时契约指 OS 面向 Agent 明确提供的交互约束与反馈规范。本章把它拆成三个维度: 更快的响应时间、结构化错误机制、面向 Prompt 注入等风险的安全信号。Prompt 注入的正式定义归 S7 展开; 本章只把它作为运行时契约需要覆盖的安全需求。运行时契约的作用是减少 Agent 用试错补环境、用推理猜错误、用经验模拟人工判断的成本。

这一定义背后的工程含义很硬。第一, 响应时间不能只服务人类耐心, Agent 的每次等待都会阻塞后续推理与工具链。第二, 错误信息不能只给人看, Agent 更需要机器可解析的错误类型、原因、影响范围和建议动作。第三, 安全风险不能只依赖人类直觉, 因为 Prompt 注入、恶意命令诱导和上下文污染会进入 Agent 的决策材料。

移动 OS 类比的准确读法

讲者用 iOS 和 Android 类比 Agent 时代的 OS 变化。触屏时代需要新的交互抽象、权限模型、应用生命周期和输入方式; Agent 时代也需要新的系统交互抽象、错误反馈、安全边界和任务运行机制。类比的重点是“交互主体变化带来系统契约变化”, 这个类比不能被读成把服务器操作系统直接改造成手机系统。



图 4: 人类用户与 Agent 用户围绕同一套 OS 能力形成不同交互闭环: Agent 更依赖毫秒级响应、结构化错误码、自愈机制和 Prompt 注入防护, 因此需要新的运行时契约。

Python 部署案例因此具有系统诊断价值。它暴露出一个系统问题: 传统 OS 把大量上下文留给人类感知和经验判断, Agent 会把这些隐式成本变成 token、调用轮次和安全暴露面。要降低这类成本, 后续章节需要回答 ANOLISA 处在整个 Agent 基础设施的哪一层, 以及它怎样通过三层架构把运行时契约落到系统能力上。

2.4 本章小结

本章把一个 5 分钟 Python 服务部署任务拆成了可计算的模型: Agent 用 14 轮完成任务, 其中 13 轮集中在目录、权限和冲突处理; 约 80% token 消耗在环境探索与试错; 内部观察显示调用轮次约为人类的 3 到 5 倍。由此得到的关键定义是运行时契约: OS 需要给 Agent 更快响应、结构化错误和安全风险信号。下一章将沿着这个契约需求进入 ANOLISA 的定位与三层架构。

3 ANOLISA 的定位与三层架构

3.1 它服务谁: Agent 与托管平台

上一章已经把问题压到“运行时契约”上: Agent 需要更快响应、结构化错误和明确的安全信号。讲者在进入产品架构前先暂停一步, 明确 ANOLISA 的位置。这一步很重要, 因为 Agent 基础设施容易被混成三类东西: 模型能力、Agent 平台能力、操作系统底座能力。三者相互配合, 职责却不同。

现有 Agent 平台更关注模型、编排、产品入口和业务流程。ANOLISA 的位置更低一层, 它面向 Agent 及 Agent 托管平台提供操作系统底座服务。这里的“底座”可以理解成道路、水电和消防系统: 上层平台像园区里的办公楼, 模型像员工和设备; 底座负责让环境可运行、可观测、可约束、可扩展。Agent 调用工具、读目录、处理权限、看日志、启动服务时, 真正承接这些动作的是 OS 层。



图 5: ANOLISA 的定位是为 Agent 与 Agent 托管平台提供操作系统底座服务，位于上层 Agent 平台能力与底层传统 Linux 能力之间。

核心定义：操作系统底座

操作系统底座指 ANOLISA 在 Agent 平台与传统 Linux 之间提供的系统级能力层。它服务的对象有两类：一类是直接运行任务的 Agent，另一类是托管、调度和管理 Agent 的平台。它的核心价值是把底层 OS 的复杂状态整理成 Agent 更容易调用、更容易被平台治理的能力。

这个定位还带来一个容易忽略的边界：ANOLISA 并不替代上层 Agent 平台的业务编排，也不替模型做推理决策。它处理的是系统侧约束，例如 Agent 工作负载怎样被内核理解，OS 能力怎样被封装成高级服务，人和 Agent 怎样通过统一入口访问这些服务。换成 Python 类比，模型和平台像业务代码与框架，ANOLISA 更像运行时、标准库和系统调用旁边的一层能力适配器。

3.2 叠加传统 Linux 的升级路线

讲者接着回应 IT 工程师和决策者最自然的顾虑：Agent 时代是否又要重新选择一套 OS。传统企业迁移操作系统时，常见成本来自版本选择、兼容性验证、工程化替换和长周期灰度。讲者提到 CentOS 替换这类历史场景，目的就是提醒读者：如果 Agentic OS 意味着大规模换底层发行版，落地阻力会非常高。

ANOLISA 给出的路线是叠加。它已经适配传统 Linux 发行版，包括阿里巴巴 Cloud Linux、Ubuntu 以及其他 Linux 操作系统。用户无需把底层 OS 整体替换掉，再经过一轮长周期验证；ANOLISA 在传统操作系统之上增加一层 Agent 能力，使原来的 OS 获得面向 Agent 的运行能力。

为什么“叠加”是关键工程选择

“叠加”降低的是迁移决策成本。企业原来已经有镜像、运维脚本、权限体系、监控规则和合规流程，如果底层发行版整体换掉，所有这些资产都要重新验证。ANOLISA 的叠加路线更像给既有道路加自动驾驶标识、车路协同信号和安全护栏，让 Agent 能读懂环境，同时保留原有道路体系。

核心定义：Agentic OS

Agentic OS 指传统 OS 叠加 ANOLISA 后形成的面向 Agent 的升级路线。它的重点从单个发行版名称转向系统设计目标：OS 开始把 Agent 的持续任务、工具调用、环境理解、资源消耗和安全边界纳入设计。

这个定义也解释了为什么讲者把 ANOLISA 叫作“传统 OS 到 Agentic OS 的升级路线”。升级的对象是 OS 对 Agent 的服务方式：从默认服务人类交互，扩展到服务 Agent 的高频、自动化、长时间运行任务。

3.3 系统优化层、运行时层、封装交互层

有了定位和兼容路线，架构大图就可以读成三层递进：底层让内核理解 Agent 工作负载，中层把 OS 能力转译成 Agent 可调用服务，上层给人和 Agent 提供统一交互入口。三层共同构成一条从“系统感知”到“能力暴露”再到“入口调用”的链路。



图 6: 三层架构图展示了 ANOLISA 的升级路线：在 Alibaba Cloud Linux、Ubuntu 等传统 OS 之上叠加系统优化层、运行时层和封装交互层，形成面向 Agent 的 Agentic OS。

三层架构总览

ANOLISA 的三层架构包括系统优化层、运行时层和封装交互层。系统优化层解决“内核怎样感知 Agent 工作负载”；运行时层解决“OS 能力怎样变成 Agent 可调用的高级服务”；封装交互层解决“人和 Agent 通过什么统一入口使用这些服务”。

第一层是系统优化层。它位于架构底部，目标是让内核感知 Agent 工作负载。传统 Linux 面向通用算力负载已经非常成熟，可 Agent 负载有新的形态：多轮工具调用、密集日志读取、长时间任务、并发 Agent 会话、频繁环境探测、对结构化反馈的强依赖。系统优化层的职责，是让底层系统能识别这些负载特征，并为后续的内存、调度、隔离和性能优化留下抓手。具体内核与性能细节会在后文运行时性能章节展开，本章只规定它在三层架构中的位置。

第二层是运行时层，也是讲者称为“核心层”的部分。它像 Agent 的服务工具箱，把底层 OS 能力转化为更高级的服务，再暴露给 Agent 调用。讲者在这里点出四类运行时能力：token 压缩、插件运行时增强、Agent 可观测、Agent 安全防护。这个列表的关键不在于数量，而在于转译方向：底层 OS 原本提供文件、进程、权限、网络、日志和资源管理；运行时层把这些能力包装成 Agent 能直接理解和服务的服务。

第三层是封装交互层。它面向人和 Agent 提供统一交互平台。人类交互越来越语义化，Agent 交互也朝意图和目标驱动演进，因此这一层会把底层能力收束到 Agent Native Shell 这样的入口上。对人来说，它减少命令、参数和管道记忆；对 Agent 来说，它减少把目标翻译成低层命令的中间成本。后续章节会专门展开 Copilot Shell 的调用形态，本章只说明它属于封装交互层的入口方向。

边界提醒：三层架构不能读成三张功能清单

如果只把三层看成“底层、中层、上层”的静态清单，就会漏掉 ANOLISA 的主线。真正的链路是：系统优化层让 OS 看见 Agent 负载，运行时层把 OS 能力转成 Agent 服务，封装交互层让这些服务被人和 Agent 以统一方式调用。少掉任意一层，运行时契约都会断在某个环节。

3.4 本章小结

本章完成了 ANOLISA 的架构定位。第一，ANOLISA 是面向 Agent 和 Agent 托管平台的操作系统底座，位置低于模型产品和平台编排。第二，它走兼容传统 Linux 的叠加路线，可在阿里巴巴 Cloud Linux、Ubuntu 等既有发行版之上形成 Agentic OS。第三，它的三层架构从系统优化层开始，让内核感知 Agent 工作负载；经由运行时层，把 OS 能力转化为 Agent 可调用服务；最后进入封装交互层，为人和 Agent 提供统一入口。下一章将沿着运行时层最先展开的能力，进入少想、少装、少传这三条 token 优化路径。

4 从四项优势到 Token 优化：少想、少装、少传

4.1 四项优势的总览

上一章给出了三层架构，讲者随后把架构能力具象化为四项优势。这里的“四项优势”从 OS 底座视角展开，避免落入产品宣传词堆叠；每一项都对应 Agent 运行中的一类硬成本。

第一项是 token 优化。讲者给出总效果：通过少想、少装、少传三个维度，ANOLISA 能让同样任务节省超过 30% 的 token 浪费。第二项是 Agent 管理与 Skill 生态，包括 Agent Native 的默认



图 7: ANOLISA 的四项核心优势总览：Token 优化、Agent 管理与 Skill 生态、运行时性能优化，以及作为 OS 层兜底的内生安全能力。

视频时间：00:08:28–00:09:59

shell、高频 Skill 接入，以及 S8 会说明的龙蜥 Skill Hub 生态承载。第三项是运行时性能优化，包括 Python、Node.js 的系统层优化，以及内存和调度层面的工作。第四项是安全，OS 层承担兜底角色，通过后文会展开的安全防护机制阻断 Agent 风险。

四项优势的读法

四项优势可以按 Agent 执行链来读：token 优化降低交互与上下文成本；管理与 Skill 生态降低能力发现成本；运行时性能优化降低资源与并发成本；安全降低自主执行带来的风险成本。本章只深入第一项 token 优化，其余三项由后续章节展开。

这一段的教学重点是顺序。讲者先列出四项优势，再把最长篇幅给 token 优化，因为前面 Python 部署案例已经证明，Agent 的账单问题首先暴露和环境探索和上下文使用上。换句话说，token 优化不能只当作单独的省钱技巧；它是 ANOLISA 运行时层最容易被量化的一条能力线。

4.2 应用层优化与 OS 层优化的叠加

讲者随后把 ANOLISA 的 30% token 降低放进业界背景中。行业里已经有应用层优化：记忆层 token 优化有团队声称能达到 3 到 4 倍成本优化，Prompt 工程优化也有 70% 到 80% 的效果说法。这里必须把指标层次分清。记忆层和 Prompt 工程改的是应用如何组织上下文、保存历史和表达任务；ANOLISA 讨论的是 OS 层如何改变 Agent 看到环境、装入上下文和传输信息的方式。

指标边界：30%、3 到 4 倍、70% 到 80% 不能混用

本讲中的 ANOLISA 指标是同样任务下 token 消耗降低超过或约 30%，并且来自 OS 层的少想、少装、少传路径。3 到 4 倍是讲者转述的记忆层优化量级，70% 到 80% 是 Prompt 工程优化效果的业界说法。三者位于不同优化层，不能相互替换，也不能简单相加。

为了把“降低 30%”说清楚，可以先用一句话理解：同一个任务，在基线环境里消耗一批 token；升级 ANOLISA 后消耗更少 token；两者差值除以基线消耗，就是节省率。

$$s_{\text{token}} = \frac{T_{\text{base}} - T_{\text{ANOLISA}}}{T_{\text{base}}} \approx 30\%$$

符号说明：

- s_{token} ：token 节省率。
- T_{base} ：基线环境中完成同一任务的 token 消耗。
- T_{ANOLISA} ：升级 ANOLISA 后完成同一任务的 token 消耗。
- $\approx 30\%$ ：讲者在总结处给出的约 30% 效果；前面总览处使用“超过 30%”表达。

OS 层优化与应用层优化的关系可以理解为叠加。应用层减少“任务怎么表达”和“历史怎么记”；OS 层减少“环境怎么被探索”“能力怎么被找到”“结果怎么被传回”。两层一起工作时，Agent 的任务链会同时受益于更好的提示词、更短的记忆、更结构化的系统能力和更少的冗余传输。

4.3 少想：OS Skill 与环境地图

少想的核心含义是减少 Agent 为了搞懂环境而进行的探索式推理。上一章和第二章已经看到，Agent 会把目录结构、权限状态、冲突处理和部署经验重新摸一遍。少想要做的事，是把这些系统知识提前沉淀在 OS 里，让 Agent 面对任务时少猜、少试、少绕路。

核心定义：少想与 OS Skill

少想指通过 OS Skill、结构化操作说明书和操作系统环境地图，减少 Agent 的探索式推理。OS Skill 是内置在操作系统中的结构化能力与经验，它把阿里云操作系统团队沉淀的知识转化成 Agent 可调用、可理解、可复用的系统能力。

讲者用了一个很直观的说法：Agentic OS 里像是已经有一位阿里云操作系统专家。这个“专家”的职责是告诉 Agent：这个环境有什么、常见操作怎么做、哪些路径高概率正确、哪些错误需要怎样处理。对 Agent 来说，OS Skill 就像一张城市地图加一套交通规则；没有地图时，它只能沿路问人和试错，有地图时，它可以直接规划路径。

4.4 少装：Skill 文件系统与上下文裁剪

少装解决的是上下文装入量问题。Agent 每一轮并不需要看完整系统状态，也不需要把所有 Skill、文件说明、历史记录和错误材料全部塞进上下文。装得越多，输入 token 越高，模型注意力也越容易被无关信息分散。

讲者提到的 Skill 文件系统通过编译优化、索引和上下文优化，只暴露 Agent 在当前轮真正关心的信息。这个机制的重点是“按需装入”：先把系统能力和经验做成可索引、可定位、可裁剪的结构，再根据当前任务切出最相关的上下文片段。

用文件系统理解 Skill 文件系统

普通文件系统让程序按路径定位文件，避免一次性读取整块磁盘。Skill 文件系统的直觉类似：它让 Agent 按任务需要定位 Skill 与说明，避免一次性装入所有系统知识。区别在于，Skill 文件系统服务的是上下文窗口和工具调用，而普通文件系统服务的是字节存储。

少想和少装经常一起出现，但作用点不同。少想减少 “Agent 需要推理多少步才能知道该做什么”；少装减少 “Agent 每一轮需要带多少材料才能做这一步”。前者减少探索链，后者减少上下文体积。

4.5 少传：输入输出精简

少传处理传输层面的冗余。Agent 与 OS 交互时，输入可能包含重复目标、冗长环境说明和无关历史；输出可能包含长日志、重复字段、低价值提示和人类阅读友好的装饰文本。对人类来说，这些内容有时还能帮助扫读；对 Agent 来说，很多内容会直接变成 token 成本。

少传使用输入和输出的自动化精简技术，把冗余传输内容压下去。用普通话说，它的目标是在保持信息可用的前提下，让传给 Agent、从 Agent 收回的信息更贴近当前任务的必要集合。少想、少装、少传三条路径可以用一个教学拆解表示：token 浪费主要来自探索推理、上下文装入和输入输出传输；ANOLISA 分别压缩这三块。

$$T_{\text{waste}} \approx T_{\text{reasoning}} + T_{\text{loaded}} + T_{\text{sent}} + T_{\text{received}}$$

符号说明：

- T_{waste} ：用于解释 token 浪费来源的教学拆解量。
- $T_{\text{reasoning}}$ ：Agent 为探索环境、猜测路径和试错消耗的推理 token。
- T_{loaded} ：被装入上下文窗口的 Skill、说明、日志和环境材料所消耗的 token。
- T_{sent} ：发送给 Agent 或模型侧的输入 token。
- T_{received} ：从 Agent 或模型侧返回并继续参与后续处理的输出 token。
- $\approx$ ：表示这是教学近似，用来区分三条优化路径，并非讲者给出的精确计量公式。

按照这个拆解，少想主要压低 $T_{\text{reasoning}}$ ，少装主要压低 T_{loaded} ，少传主要压低 T_{sent} 和 T_{received} 。这也解释了讲者那句总结：“同样的任务，token 消耗降低 30%；不换模型，只升级 OS。”模型仍然是同一个模型，变化发生在 OS 提供给 Agent 的环境地图、上下文切片和传输内容上。

4.6 Dashboard 证明节省发生在哪里

讲者最后强调，token 节省需要可观测入口。ANOLISA Dashboard 的作用，是展示整体节省了多少 token，也展示 token 节省发生在什么地方。这个点非常关键，因为没有可观测性，token 优化就会变成主观感受：看起来少了几轮、看起来日志短了一点、看起来成本低了一些。Dashboard 把这些感受变成可以检查的运行证据。

核心定义：ANOLISA Dashboard

ANOLISA Dashboard 是 token 节省的可观测入口。它用于观察整体 token 降低效果，并定位节省来自少想、少装、少传中的哪些环节。本章中的 Dashboard 专指 token 优化观测，安全可观测会在后续安全章节按安全语境展开。

Dashboard 对工程团队还有一个现实意义：它能防止优化讨论停在口号层。少想是否真的减少环境探索，少装是否真的裁掉无关上下文，少传是否真的减少输入输出冗余，都需要在数据面上看

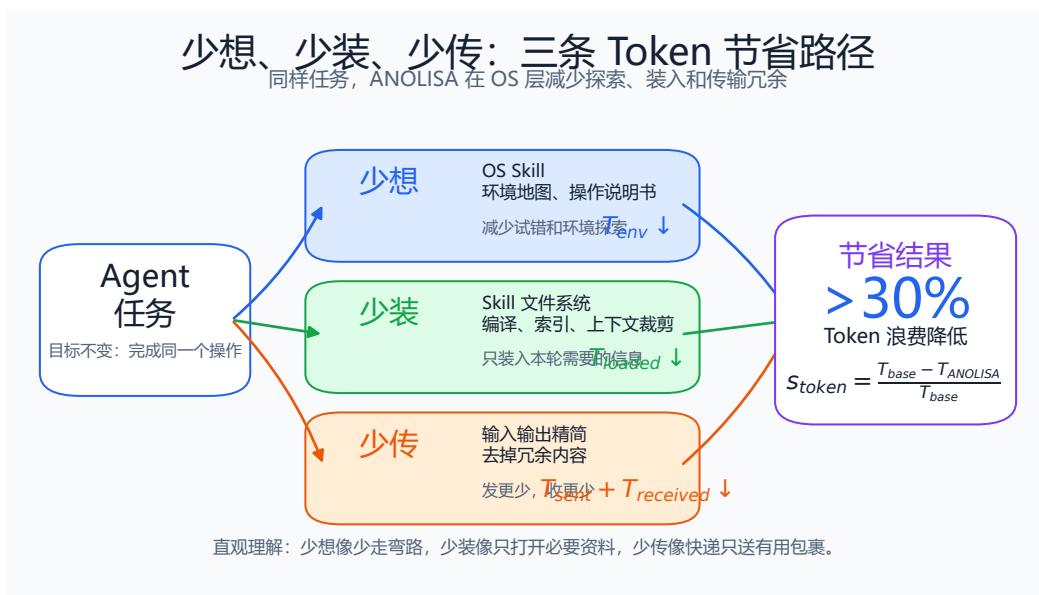


图 8: 少想、少装、少传的教学示意图：少想减少环境探索，少装减少本轮上下文装入，少传减少输入输出中的冗余传输。

视频时间：00:10:44-00:12:23；原区间未出现可单独表达三路径的完整大图，因此按讲者机制说明重绘。



图 9: ANOLISA Dashboard 展示 Token 节省效果：同样任务降低约 30% Token 消耗，并把节省位置拆到系统级优化、Skill 文件系统、压缩与 OS Skill 等来源。

视频时间：00:12:33-00:12:59

见。只有能定位节省发生的位置，团队才能决定下一步该补 OS Skill、改索引策略，还是压缩 I/O 返回格式。

4.7 本章小结

本章从四项优势进入 token 优化。ANOLISA 的四项优势分别对应 token、管理与 Skill 生态、运行时性能和安全四类成本，其中 token 优化通过少想、少装、少传三条路径展开。少想依靠 OS Skill、结构化说明书和环境地图减少探索式推理；少装依靠 Skill 文件系统、编译优化、索引和上下文裁剪减少装入量；少传依靠输入输出自动精简减少冗余传输。讲者给出的 ANOLISA 效果是同样任务 token 消耗降低超过或约 30%，这个 OS 层指标与记忆层 3 到 4 倍优化、Prompt 工程 70% 到 80% 优化位于不同层级。下一章会转向 Copilot Shell：这些 OS 能力最终需要一个意图式调用表面，让人和 Agent 能把能力真正用起来。

5 Copilot Shell：把 OS 能力变成意图式入口

5.1 从语法式 CLI 到意图式交互

上一章已经说明，ANOLISA 通过少想、少装、少传减少 Agent 与 OS 交互时的 token 浪费。接下来的问题更靠近使用入口：底层 OS 已经沉淀了能力，Agent 和人怎样稳定地调用这些能力？讲者在 00:13:02-00:13:50 把这一层引到 Shell。图像核验确认页面标题使用 Copilot Shell；正文把它解释为一种 Agent Native Shell，也就是面向 Agent 的意图式 OS gateway。

传统 CLI 的优势是精确，代价是语法负担重。用户需要记住命令名、参数、选项顺序、路径规则、管道组合和错误提示的含义。对熟练工程师来说，这些知识会逐渐变成肌肉记忆；对 Agent 来说，这些知识往往会变成上下文搜索、命令试探、错误修正和重复调用。换句话说，CLI 把任务目标拆成了大量语法细节，Agent 必须先学会怎样表达动作，然后才能执行动作。

讲者用压缩文件的例子说明这种差异。人或 Agent 的真实目标可以很简单：把某个文件压缩起来。传统 CLI 要求调用者知道应使用哪类命令、怎样组织源路径和目标路径、是否需要递归、压缩格式如何选择、命令失败后如何解释返回值。这里的 tar 命令只是代表性例子，用来说明命令参数负担；本章不展开 Unix 工具教学。

CLI 语法会把 Agent 推回试错循环

Agent 面对传统 CLI 时，经常先推断命令、再拼参数、再解释错误、再改写命令。这个循环会消耗 token、时间和系统调用次数，也会放大误操作概率。前文讨论的运行时契约要求更快响应和更结构化错误，Shell 入口正是这一契约在交互层的具体落点。

意图式交互的方向是把“我要完成什么”放在入口中心。Agent 可以表达“压缩这个目录”“拉起这个服务”“给这块云盘扩容”这类目标；Agent Native Shell 负责把目标翻译成可执行的 OS 能力调用，并把执行结果以统一形式返回。这里的关键变化是：调用者不再直接承担全部命令语法组织工作，系统入口承担意图解析、能力路由、执行编排和结果结构化。



图 10: Copilot Shell 把传统 CLI 的命令、参数和管道负担前移为意图式入口：Agent 表达“做什么”，系统负责解析“怎么做”。

视频时间：00:13:02-00:13:50；屏幕标题核验为 Copilot Shell。

核心定义：Copilot Shell 与 Agent Native Shell

Copilot Shell 是画面中出现的入口名称；在机制上，它承担 Agent Native Shell 的职责。它把自然语言目标、任务意图或上层 Agent 请求，转成底层 OS 能力调用，并以一致、可理解、可继续推理的形式返回结果。

5.2 统一 gateway 如何暴露 OS 能力

讲者在 00:13:36-00:13:50 给出 Agent Native Shell 的核心定位：它提供一个统一 gateway 层，把底层操作系统能力暴露给 Agent。统一 gateway 的价值在于稳定边界。上层入口可以不同：自研 Agent 可以调用，OpenCloud（按字幕称呼）界面可以调用，其他 Agent 入口也可以调用；底层输出需要保持一致，避免每个入口都重新适配一套 OS 细节。

核心定义：统一 gateway

统一 gateway 是 Agent Native Shell 暴露 OS 能力的接口层。它把底层 OS 的命令、服务、状态和错误转成一致输出，供自研 Agent、OpenCloud（按字幕称呼）以及其他上层入口调用。S5 只定义这一 gateway 的交互职责，不重新定义 S3 的三层架构。

可以把统一 gateway 的工作流拆成五步。第一步是接收意图：输入可能来自自然语言，也可能来自上层 Agent 的结构化请求。第二步是规整目标：系统需要识别对象、动作、约束和风险点，例如“哪个服务”“哪块云盘”“扩容到什么范围”。第三步是路由能力：gateway 把目标映射到 OS 能力、Skill 或云产品相关操作。第四步是执行与观察：底层完成动作，并收集返回码、日志、状态变化和错误信息。第五步是统一输出：把结果包装成 Agent 能继续使用的观察，避免把原始命令碎片直接丢回去。

用“前台窗口”和“后台能力”理解 gateway

对人来说, Shell 像一个前台窗口; 对 Agent 来说, Shell 更像一个 API 化的任务入口。统一 gateway 把后台 OS 能力整理成稳定服务, 让不同前台都能拿到相同语义的结果。这样一来, 上层入口可以变化, 底层能力的表达方式保持一致。

讲者还强调, 这个 Shell 也给人使用。原因很现实: 企业里仍然存在需要登录到 OS 层处理问题的场景。面向人的入口和面向 Agent 的入口共用同一层 gateway, 可以减少两套操作路径之间的偏差。人看到的结果、Agent 接收的结果、上层平台记录的结果越一致, 排查和治理越容易。

5.3 Demo 1: 一句自然语言拉起服务

第一个 demo 位于 00:14:04-00:14:36。讲者说明现场字幕看不清, 因此本章只依据口播内容概括。这个 demo 的任务是通过一句自然语言拉起一个服务, 并配置好与钉钉的连接。重点不在服务名本身, 重点在任务链: 一个自然语言目标驱动了多步骤 OS 操作。

这类任务如果交给传统 CLI, 通常需要拆成若干步: 确认服务环境、准备配置、启动进程、接入外部系统、检查状态、处理错误。Agent Native Shell 把这些步骤收束到一个意图入口。调用者说出目标, Shell 负责把目标落到 OS 能力上, 并把执行结果返回给调用者。这里的“自然语言”并不意味着系统只做文本聊天, 它承担的是任务描述层; 真正落地仍然依靠 OS 能力、运行时层服务和结构化观察。



图 11: 自然语言服务拉起 demo: 左侧终端展示多步骤操作轨迹, 右侧说明 Copilot Shell 通过自然语言与 bash 双模交互承接 Agent 操作。

视频时间: 00:14:04-00:14:36; 终端细节字号较小, 图注只概括可见流程。

Demo 1 证明了什么

自然语言拉起服务的 demo 证明, Agent Native Shell 可以把一句任务目标转成多步骤 OS 操作。它降低的是入口复杂度: 调用者无需逐条拼接命令, Agent 也无需反复猜测命令语法和环境细节。

这个 demo 与前一章的 token 优化直接相连。少想让 Agent 少做环境探索，少装让 Agent 少带无关材料，少传让 I/O 更精简；Agent Native Shell 则把这些底层优化组织成一个可调用入口。没有入口，能力会停留在系统内部；有了入口，能力才能进入人和 Agent 的实际任务流。

5.4 Demo 2: 通过 Agent 接口完成云盘扩容

第二个 demo 位于 00:14:39–00:15:31。讲者描述的场景是：在 OpenCloud（按字幕称呼）交互界面里，人告诉一个 Agent，希望给操作系统的云盘做扩容操作；随后可以看到 Agent 怎样调用操作系统、理解这个任务并实施扩容。这里仍然需要保持表述克制：OpenCloud 只作为字幕中的界面名出现，本章不扩展它的产品背景。

云盘扩容是一个比“启动服务”更能体现 gateway 价值的例子。它牵涉资源对象、容量变化、执行权限、系统状态刷新和结果确认。对上层 Agent 来说，它希望表达的是“扩容这块云盘”；对 OS 来说，扩容需要落到具体设备、文件系统、挂载状态和云资源操作上。统一 gateway 的职责，是在这两层之间提供稳定翻译。



图 12: 云盘扩容 demo：OpenCloud 风格界面作为上层入口，仍通过统一 gateway 输出操作结果，说明不同 Agent 入口可以复用同一套 OS 能力。

视频时间：00:14:39–00:15:31；OpenCloud 名称按字幕和弱界面线索保留，具体字段不可清晰辨读。

这段 demo 的收束句很关键：无论入口来自自研 Agent，还是 OpenCloud（按字幕称呼），或是其他 Agent 入口，整体 gateway 的统一输出都应保持一致。这里的“一致”至少包含三层含义：同类请求使用同类能力路由；同类结果采用同类结构返回；同类错误给出可继续处理的诊断线索。这样，上层系统不会因为入口不同而得到语义不一致的 OS 反馈。

不要把统一 gateway 理解成单一界面

统一 gateway 的重点是输出语义一致，而非所有用户都使用同一个屏幕。自然语言入口、Agent 接口和云产品界面可以并存；只要它们调用的是同一层 OS gateway，治理、审计、诊断和后续自动化才有稳定基础。

5.5 本章小结

本章把 OS 能力的调用入口从传统 CLI 推进到意图式 Shell。传统 CLI 要求调用者掌握命令、参数和管道，Agent 因此容易陷入命令推断与试错循环。Agent Native Shell 提供意图式入口，把任务目标转成 OS 能力调用；统一 gateway 则把底层 OS 能力以一致输出暴露给自研 Agent、OpenCloud（按字幕称呼）和其他入口。两个 demo 分别证明了自然语言可以拉起服务并配置钉钉连接，以及 Agent 可以通过 OpenCloud（按字幕称呼）界面驱动云盘扩容。入口统一之后，下一个瓶颈会落到多个 Agent 同时运行时的并发、内存与调度承压。

6 高密 Agent 运行时：小规格实例上的并发、内存与调度

6.1 每名员工 10 到 100 个 Agent 的负载假设

上一章讲的是入口：Agent Native Shell 让人和 Agent 以意图方式调用 OS 能力。讲者随后在 00:15:36–00:16:59 转到运行时压力：企业未来的每名员工可能配备 10 到 100 个 Agent，一起完成日常工作。这个判断把问题从“单个 Agent 如何调用 OS”推到“很多 Agent 同时跑在同一台服务器上时，OS 怎样承压”。

这个量级可以用一个简单公式表示：

$$N_{\text{total}} = E \times N_{\text{agent/employee}}, \quad N_{\text{agent/employee}} = 10-100$$

符号说明：

- N_{total} ：企业侧需要承载的 Agent 总数。
- E ：员工数量。
- $N_{\text{agent/employee}}$ ：每名员工配备的 Agent 数量，讲者给出的假设范围是 10 到 100。
- 10–100：用于引出高密运行时压力的负载假设，不能外推为所有企业的确定部署规模。

核心定义：高密 Agent 运行时

高密 Agent 运行时指在同一服务器或小规格实例上，同时承载大量 Agent 会话、工具调用和任务状态，并通过并发控制、内存分配、调度策略和隔离机制，让多个 Agent 长时间运行且互不干扰的 OS 运行能力。

这个公式的直觉很直接。一个员工配一个 Agent 时，系统面对的是助手级负载；一个员工配 10 到 100 个 Agent 时，系统面对的是小型任务群。若很多员工同时工作，Agent 数量会按乘法增长。Agent 数量增长后，压力首先落到三个 OS 维度：并发会话是否能撑住，内存是否能合理分配，调度是否能避免互相挤占。

6.2 小规格实例上的三类压力

讲者特别提到，ANOLISA 对阿里云上的轻量服务器和小规格实例做了针对性优化。这个背景很重要：小规格实例的资源余量有限，CPU、内存、I/O、网络 and 进程数都更容易触顶。单个 Agent 看起来轻量，几十个 Agent 加上工具进程、日志、连接和缓存后，就会形成明显的系统压力。

第一类压力是并发会话。Agent 的“会话”通常不只是一次请求，它可能包含任务状态、工具调用、环境观察、错误重试和上下文记录。多个 Agent 同时运行时，OS 需要处理并行的进程、线程、文件描述符、网络连接和子任务队列。并发处理做得粗糙，就会出现某个 Agent 长时间占用资源，其他 Agent 等待或超时的情况。

第二类压力是内存分配。Agent 运行会带来模型侧上下文、工具进程、脚本运行环境、缓存、日志缓冲和中间文件。即使模型推理发生在远端，本地运行时仍然要承载大量状态。可以用一个容量约束来理解：

$$M_{\text{host}} \geq M_{\text{os}} + \sum_{i=1}^N (M_{\text{agent},i} + M_{\text{tool},i} + M_{\text{cache},i})$$

符号说明：

- M_{host} ：主机或实例可用内存容量。
- M_{os} ：操作系统和基础服务自身需要的内存。
- N ：同一服务器上同时运行的 Agent 数量。
- $M_{\text{agent},i}$ ：第 i 个 Agent 的会话状态和运行框架占用。
- $M_{\text{tool},i}$ ：第 i 个 Agent 调用工具、脚本或子进程产生的内存占用。
- $M_{\text{cache},i}$ ：第 i 个 Agent 相关缓存、日志缓冲和中间状态占用。

第三类压力是调度。Agent 任务往往有明显的长尾：有的任务只是查询状态，有的任务会拉起服务，有的任务会等待外部系统，有的任务会进入多轮修复。OS 调度需要在 CPU 时间、I/O 等待、网络请求和任务优先级之间做平衡。若调度只看平均负载，突发任务和长时间任务会互相拖慢。

平均负载会掩盖 Agent 运行时的尖峰

高密 Agent 场景里的风险经常来自瞬时并发和长尾任务，而非全天平均值。几十个 Agent 同时拉起工具、写日志、访问网络或重试失败步骤时，小规格实例会先在内存、文件描述符、I/O 队列或调度延迟上暴露问题。

6.3 高密部署与互不干扰

讲者给出的第一个优化方向是高密部署：多 Agent 同时运行，并且互不干扰。这里的互不干扰需要从资源、状态和故障三个层次理解。资源层面，一个 Agent 不能无限占用 CPU、内存、I/O 或网络连接；状态层面，一个 Agent 的文件、日志、临时目录和环境变量不能污染另一个 Agent；故障层面，一个 Agent 的崩溃、死循环或异常重试不能拖垮同机其他任务。

这正好呼应 S3 中定义的系统优化层：让内核感知 Agent 工作负载。通用服务器负载通常用进程、线程、I/O、网络 and 内存指标描述；Agent 负载还带有任务链、工具链、上下文链和重试链。内核与运行时层若能识别这些链条，就可以更细地做限额、隔离、优先级调整和异常处置。

用“共享办公室”理解互不干扰

多个 Agent 共用一台服务器，像多人共用一间办公室。每个人都需要桌面、电源、网络 and 会议室；如果有人长时间占满会议室，其他人就会被迫等待。高密运行时要做的是给每个 Agent 分配边界、排队规则和异常处理通道，让共享资源被有序使用。

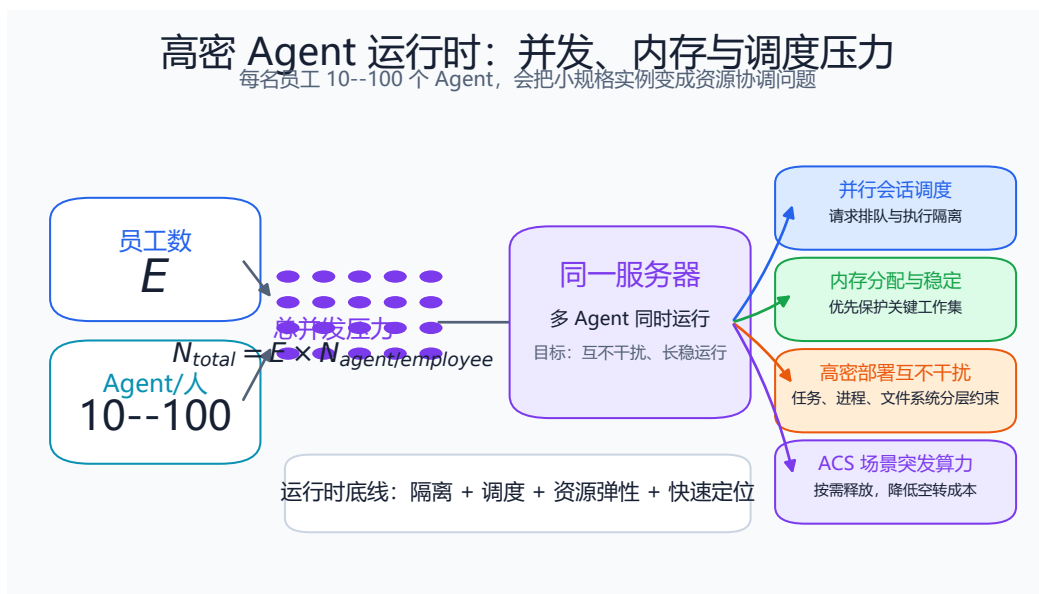


图 13: 高密 Agent 运行时示意图：每名员工 10-100 个 Agent 会放大同一服务器上的并发、隔离、内存、调度和 ACS 突发算力压力。

视频时间：00:15:36-00:16:59；原区间性能页未完整表达负载假设到调度压力的链条，因此按讲者机制说明重绘。

调度策略的目标是让资源分配符合任务优先级、风险边界和系统稳定性。短任务可以快速完成，长任务需要被限速或迁移，高风险任务需要额外审查，高消耗工具需要配额。这样，系统才能在同一台机器上承载多个 Agent 的持续工作。

6.4 ACS 场景、突发算力与长稳运行

讲者接着提到 ACS 场景下的突发算力按需释放。这里没有足够上下文核验 ACS 全称，因此正文只写 ACS 场景。它表达的运行时需求很清楚：Agent 工作负载并非一直平稳，有些时刻需要突然增加算力，有些时刻又需要释放资源以节约成本。高密运行时需要识别这种波峰波谷，让资源在需要时可用，在空闲时回收。

ACS 场景中的突发算力

在 ACS 场景中，本章只保留讲者明确表达的机制方向：突发算力需要按需释放，并且从用户节约成本的角度考虑。具体产品全称、资源形态和调度算法需要后续图像或公开资料核验后再展开。

突发算力释放可以理解为运行时的弹性闸门。Agent 群平时维持低资源占用；当多个任务同时进入执行阶段，系统临时释放更多计算资源；当任务结束，资源回落到较低水平。这个过程如果缺少调度约束，就会出现两类问题：资源释放不足导致任务排队，资源释放过多导致成本失控。ANOLISA 在这里强调的是 OS 视角的平衡：既要让 Agent 有能力完成任务，又要让用户不为短暂峰值长期付费。

最后两个方向是长时间运行不崩溃和快速定位问题。长稳运行关注的是资源泄漏、僵尸进程、异常重试、日志膨胀、连接堆积和状态漂移。快速定位关注的是当任务失败时，系统能否迅速指出问题发生在哪个 Agent、哪次工具调用、哪个资源对象、哪个系统指标上。没有快速定位，高密部署会把排查难度放大：同一台服务器上 Agent 越多，故障线索越容易混在一起。

高密运行时不能只追求“跑得多”

高密部署的目标包括并发密度、互不干扰、弹性资源、长稳运行和快速诊断。若只提升同机 Agent 数量，却缺少隔离、配额、事件日志和健康检查，系统会把单个 Agent 的小故障放大成整台实例的运行风险。

6.5 本章小结

本章把“每名员工 10 到 100 个 Agent”的负载假设转成 OS 层压力模型。Agent 数量按员工规模放大后，小规格实例会在并发会话、内存分配和调度策略上承受明显压力。高密 Agent 运行时的任务，是让多个 Agent 在同一服务器上同时运行并互不干扰；在 ACS 场景中，它还要让突发算力按需释放，兼顾执行能力与成本控制。长时间运行不崩溃和快速定位问题，是高密部署进入生产可用状态的关键条件。性能与稳定性把 Agent 放上生产轨道，下一步要回答它们能触达哪些资源、越界时由谁拦截，这就进入安全边界问题。

7 Agent 安全：执行前、执行中、OS 层兜底

7.1 Agent 安全为什么决定自主执行边界

上一章从高密 Agent 运行时讲到并发、内存、调度和长稳运行。到这里，问题自然转向安全：一台机器能同时跑很多 Agent 只是前提，企业真正关心的是这些 Agent 能跑到多远、能调用哪些资源、出错后能否追责。讲者在 00:17:21–00:17:26 给出的判断很直接：Agent 安全会决定 Agent 是否可以自主地去做一些工作。

这句话的含义需要说透。人类操作系统用户通常以一次次命令、一次次点击的方式推进任务，风险边界可以通过账号、目录权限、网络策略、审批流程逐层约束。Agent 的工作方式更像一个持续行动者：它会读取上下文、生成计划、调用工具、观察结果、继续修正动作。只要给它更大的自主权，它就会跨过更多中间步骤；只要中间步骤更多，安全系统需要管理的状态空间就会扩大。

可以把 Agent 的自主执行边界写成一个简单的教学模型：

$$B_{\text{auto}} = \{a \mid R(a) \leq \tau, S(a) \subseteq P, \text{audit}(L(a)) = 1\}$$

其中：

- a ：Agent 准备执行的一个动作，例如扫描代码、修改配置、调用云资源接口。
- $R(a)$ ：动作 a 的风险估计，风险来源包括权限提升、数据外传、服务中断和合规违规。
- τ ：企业允许 Agent 自动执行的风险阈值，阈值越低，Agent 需要请求人工确认的动作越多。
- $S(a)$ ：动作 a 会触达的系统资源集合，例如文件、进程、网络端点、云产品 API。
- P ：当前账号、任务和运行环境允许触达的资源边界。
- $L(a)$ ：动作 a 产生的日志与证据链，能够支持事后复盘、合规审计和责任界定。
- $\text{audit}(L(a)) = 1$ ：日志与证据链满足审计要求，能说明谁授权、谁执行、触达了什么资源以及结果如何。

自主权来自可控边界

Agent 安全的核心任务，是把“可以自动执行的动作集合”限定在可解释、可审计、可隔离的范围内。边界越清楚，Agent 能拿到的自主空间越大；边界越模糊，企业只能把更多动作退回人工审批。

这个模型解释了为什么安全在 Agent 时代会从“防护效果”升级为“自主执行资格”。对普通服务来说，安全系统挡住一次非法请求已经有价值。对 Agent 来说，安全系统还要回答下一层问题：如果一个任务需要十几轮工具调用，前面九轮看起来都正常，第十轮突然访问敏感目录，系统能否识别上下文变化？如果 Agent 被未信任文档诱导，系统能否判断这条自然语言指令已经改变了任务意图？这些问题直接决定 Agent 能否从辅助工具走向半自主执行者。

7.2 确定性防护模型的局限

讲者接着区分了传统安全模型和 Agent 安全模型。传统安全更依赖确定性的输入输出：请求格式固定，访问路径固定，服务边界固定，白名单、防火墙、规则引擎能够把大量已知风险挡在外面。这样的控制仍然有用，它适合处理端口、IP、域名、文件路径、系统调用、权限位这类清晰对象。

Agent 的困难在于，动作链条由 Prompt、上下文、模型输出、工具返回结果共同生成。相同的用户目标在不同上下文下可能产生不同工具序列；同一个工具序列在不同系统状态下也可能产生不同后果。讲者在 00:17:43-00:17:50 强调，每一条 prompt 在每一次执行中都可能得到不同输出。这里的关键术语是 Prompt 注入：未信任内容把额外指令塞进 Agent 的语言上下文，诱导 Agent 偏离原任务，进而触发错误工具调用、越权访问或数据泄露。

用概率形式看，Agent 输出不再是简单函数 $y = f(x)$ ，更接近条件分布：

$$y \sim P_{\theta}(y \mid p, c, o, h)$$

其中：

- y ：Agent 生成的下一步输出，可能是自然语言回复，也可能是工具调用。
- P_{θ} ：模型参数为 θ 时的输出分布。
- p ：用户 prompt 或系统 prompt。
- c ：任务上下文，例如文件内容、网页内容、历史对话和环境说明。
- o ：工具调用返回的观察结果。
- h ：已有交互历史，包括前几步计划、失败原因和修正动作。

这个公式的教学意义很简单：如果 c, o, h 会在执行过程中持续变化，安全系统只在任务入口做一次静态判断就会漏掉中间态风险。白名单可以限制“能访问哪里”，防火墙可以限制“能连到哪里”，它们难以单独判断“为什么此刻要访问这里”以及“这次访问是否仍然服务于原任务”。

只做入口过滤会留下中间态盲区

Agent 风险经常出现在执行过程中：工具返回了新的文本、日志暴露了新的路径、网页里出现了诱导性指令、Agent 为了修复错误临时扩大了搜索范围。入口过滤看不到这些后续状态，单靠一次 prompt 扫描无法覆盖完整行动链。

可以用安检做类比。传统规则像登机前检查证件和行李，能挡掉明显违禁物。Agent 的执行链还需要飞行过程中的仪表监控、机组通信记录和舱室隔离，因为风险可能在旅途中出现。ANOLISA 的安全设计正是沿着这个思路展开：执行前先拦截，执行中持续监控，最后由 OS 层隔离与安全基线兜住底线。

7.3 三层防御架构

讲者在 00:17:50-00:18:38 把 ANOLISA 的 Agent 安全归纳为三层防御架构。这三层有递进关系：第一层降低明显危险动作进入执行阶段的概率，第二层发现执行过程中的异常行为，第三层在前两层失守时限制破坏半径。



图 14: ANOLISA 的内生安全页把 Agent 防护拆成三层：执行前拦截用于代码扫描与 Prompt 扫描，执行中监控用于安全可观测、事件日志与合规审计，底层兜底用于 OS 级隔离和安全基线。右侧同时给出无感防护、成本零增长、合规可审计和即插即用等交付目标。

视频时间：00:17:00-00:18:38

7.3.1 第一层：执行前拦截

执行前拦截负责在动作真正落地之前做风险预判。讲者提到两类扫描：代码扫描和 Prompt 扫描。代码扫描面向即将运行的脚本、命令、配置变更或依赖变更，重点识别危险系统调用、敏感路径写入、外联地址、权限修改、凭据暴露等问题。Prompt 扫描面向自然语言输入与上下文内容，重点识别 Prompt 注入、越权请求、隐藏指令、数据外传诱导等问题。

执行前拦截的定义

执行前拦截，是 ANOLISA 在 Agent 动作落地之前对代码、命令、Prompt 和上下文进行风险扫描的安全层。它的目标是提前识别高风险意图，让危险动作进入阻断、降权、复核或改写流程。

这一层的价值在于成本最低。能在执行前拦下的风险，就无需等到进程启动、文件已写、网络

已连之后再补救。对 Agent 工作负载来说，执行前拦截还承担“意图校准”的任务：系统需要判断当前动作是否仍然属于原始任务。例如，部署 Python 服务时读取项目目录属于合理动作；把环境变量整体发往外部域名就是风险动作。两者都可能由同一个自然语言任务链生成，安全系统必须在动作落地前区分它们。

7.3.2 第二层：执行中监控

执行中监控负责处理入口判断无法覆盖的状态变化。讲者列出的能力包括安全可观测、结构化事件日志和合规审计。安全可观测提供实时视角：Agent 调用了什么工具、访问了哪些资源、发生了哪些失败重试、风险评分是否上升。结构化事件日志把这些行为变成可检索、可关联、可复盘的数据。合规审计则把事件证据和企业制度连接起来，让安全团队能够回答“谁授权、谁执行、触达了什么、结果怎样”。

这里要把 S4 的 ANOLISA Dashboard 和本节的安全可观测区分开。前者主要证明 token 节省发生在哪里；后者关注 Agent 执行链上的风险状态。二者都属于可观测，但观测对象不同：一个看成本，一个看安全行为。

结构化事件日志为什么重要

普通日志像散落的文字记录，排查时依赖人工搜索和经验判断。结构化事件日志把时间、Agent 身份、任务 ID、工具名、资源对象、风险标签、处置结果拆成字段。这样一来，安全团队可以按任务链聚合，也可以按资源或风险类型横向分析，审计效率会高很多。

执行中监控的难点在于判定粒度。过粗会漏掉异常动作，过细会制造大量噪声。务实做法是围绕关键动作建立事件模型：文件系统写入、权限变更、外部网络连接、云资源创建、凭据读取、跨租户访问、长时间高频重试。这些事件一旦和 Agent 的原任务不匹配，就应触发告警、限速、暂停或人工复核。

7.3.3 第三层：OS 层隔离与安全基线

讲者把第三层称为最重要的一层：在前两个层级都被突破的情况下，OS 层仍然要做安全兜底。这里的 canonical term 是 OS 层隔离与安全基线。它包括进程、文件系统、网络、权限、资源配额和系统调用层面的限制，让 Agent 即使生成了危险动作，也无法无限制地触达宿主机和云资源。

OS 层隔离与安全基线

OS 层隔离与安全基线，是 ANOLISA 安全架构的最后约束：把 Agent 的进程、权限、文件、网络和资源使用锁在预设边界内。前两层负责识别风险，第三层负责限制风险已经发生时的破坏半径。

这层防线的意义在于“承认失守可能”。Prompt 扫描可能漏掉精心伪装的注入，运行时监控可能在异常动作发生后才观察到信号，模型也可能在复杂上下文中做出意外选择。OS 层隔离要求系统即使在这些情况下仍然有硬边界：敏感目录读写需要明确授权，网络外联受到策略约束，危险系统调用被限制，资源消耗有配额，跨任务状态被隔离。

从工程上看，这一层也是 ANOLISA 作为操作系统底座的独特位置。应用层安全组件可以做 prompt 策略、模型护栏和业务规则，OS 层可以直接约束进程、文件、网络和内核资源。Agent 如果要获得更高自主性，必须接受更强的底层约束。否则企业只能把 Agent 留在低权限、低价值、低自动化的任务区间。

安全基线不能只靠模型自觉

要求模型“不要做危险动作”只能降低部分风险，无法形成硬隔离。真正的兜底需要 OS 层策略：权限最小化、资源隔离、网络边界、审计日志和异常处置共同生效。

7.4 本章小结

本章把讲者的安全段落展开成一条完整逻辑：Agent 安全决定自主执行边界；Prompt 与执行上下文的不确定性削弱了只靠确定性规则的防护模型；ANOLISA 用执行前拦截、执行中监控、OS 层隔离与安全基线形成三层防御。第一层尽早识别代码与 Prompt 风险，第二层把行动链变成可观测、可审计的事件流，第三层在前两层失守时限制破坏半径。

这一章也把前文的运行时契约推到安全维度：响应更快、错误更结构化、token 更省、高密度运行更稳，都需要安全边界来支撑。下一章进入生态与开源，讨论 ANOLISA 怎样从一套运行底座扩展为开发者、云产品和 OpenAnolis 社区共同参与的 Agent 生态。

8 总结与延伸

8.1 讲者的收束：开放底座才有生态价值

讲者在 00:18:41 之后把话题从安全转向生态。他强调，ANOLISA 走的是开放底座路线。一个 Agent 基础设施的价值，最终体现在它承载的服务和应用上：开发者能否更快接入，平台能否把底层能力稳定暴露出来，社区能否贡献更多 Skill 和运行经验。

在阿里云生态内，讲者给出两个落地点。第一，ANOLISA 已经集成到多种云产品中。第二，轻量应用服务器将全面搭载 ANOLISA，让 Agent 开发者和服务平台把它作为运行底座使用。这里要抓住接入方式这一层：应用服务器把 OS 层能力、运行时能力和交互入口预装到开发者更容易触达的位置，减少从零搭建 Agent 运行环境的门槛。

生态集成的核心含义

ANOLISA 的生态集成，是把前文的运行时契约、安全边界、统一 gateway 和 OS 能力做成可被云产品承载的底座。开发者获得的是开箱即用的运行环境，平台获得的是一致的 Agent 执行接口。

这段收束也回应了 S2 的成本痛点。Python 服务部署案例里，Agent 把大量 token 花在环境探索和试错上。生态集成如果做得足够深，开发者就无需让每个 Agent 从头摸清目录、依赖、权限、网络、日志和云资源入口。环境知识被沉淀在底座里，Agent 执行任务时可以把更多 token 用在真正的业务步骤上。

8.2 GitHub 开源与龙蜥 Skill Hub

讲者随后提到 ANOLISA 已经在 GitHub 开源，并把 OpenAnolis / 龙蜥放在开源载体的位置。按照 outline contract，这里需要把龙蜥 Skill Hub 与 S4 的 Skill 文件系统区分开。Skill 文件系统是运行时内部的上下文裁剪与装入机制；龙蜥 Skill Hub 是社区和生态层面的承载入口，用来组织、分发和共建面向 Agent 的 Skill 与经验。



图 15: “丰富的产品生态”页展示 ANOLISA 位于云产品与 Agent 应用之间: 左侧说明其作为阿里云 Agent 基础设施的一部分, 服务云上 Agent; 右侧环形图把 ANOLISA 连接到 ECS/轻量应用服务器、Agent 服务平台、Agent 开发者、龙蜥社区、计算巢/无影架构和 Agent 服务提供商。

视频时间: 00:18:41-00:19:18



图 16: 同一生态页的开源说明区明确写出 ANOLISA 已在 GitHub 上开源, 并由龙蜥社区作为开源载体承载 Skill Hub 和 Agent 生态; 这张图用于区分 S4 的 Skill 文件系统和 S8 的社区级 Skill Hub。

视频时间: 00:19:19-00:19:43

Skill Hub 与 Skill 文件系统的区别

Skill Hub 像一个社区货架，负责让开发者发现、贡献和复用 Skill。Skill 文件系统像运行时里的索引和装载机制，负责让 Agent 在一次任务中只拿到相关上下文。前者解决生态供给，后者解决执行效率。

开源在这里承担两件事。第一，它给开发者一个可检查、可贡献、可适配的入口，降低对单一平台黑盒能力的依赖。第二，它让不同云产品、不同 Agent 框架、不同运维场景的经验进入同一个 Skill 生态。Agent 基础设施如果只服务少数内置场景，很快会遇到覆盖不足的问题；开源社区可以把长尾工具、行业脚本、运维知识和安全策略逐步沉淀进来。

生态开放也会带来治理压力

Skill 越多，供应链、权限、版本兼容和安全审计压力越大。一个可用的 Skill Hub 需要明确来源、版本、权限声明、测试证据和撤回机制。否则，生态越繁荣，运行底座承受的风险也会越复杂。

8.3 UNIX 例子与接口思想

讲者最后用 UNIX 做类比。UNIX 的长期影响力来自它支撑了后来尚未出现的大量程序，留下了可组合、可扩展的接口思想。放到 Agent 时代，ANOLISA 想承担类似角色：给未来的 Agent、工具和云服务留下稳定接口，让新的应用能够跑在同一个底座上。

这个类比的关键在“接口”。接口代表一组可被长期复用的约定：进程怎样启动，文件怎样访问，权限怎样隔离，日志怎样记录，工具怎样被调用，异常怎样回传。对人类开发者来说，UNIX 的接口把程序组合变得自然；对 Agent 来说，ANOLISA 试图把 OS 能力变成 Agent 可理解、可调用、可审计的服务。

讲者给出的愿景很直接：让每一个 Agent 都跑在 ANOLISA 上。随后他补充，Alibaba Cloud Linux 4 Agent 版已经在 ECS 全地域上线，并邀请开发者体验和反馈。这里应按演讲原意理解为产品可用性信号：底座路线已经从概念介绍走向云服务器可试用阶段。

最终愿景

ANOLISA 的愿景，是把 Agent 所需的环境理解、工具调用、性能调度、安全隔离和生态 Skill 收束成同一个运行底座，让 Agent 能在云服务器和开放生态中稳定执行。

8.4 全篇综合：从成本痛点到生态承载

把全篇压缩成一条链，可以看到 ANOLISA 的论证方式非常清晰。

1. 成本痛点：Agent 执行简单任务时，会把大量 token 和轮次消耗在环境探索、权限试错和错误恢复上。
2. 运行时契约：Agent 需要更快响应、更结构化的错误、更明确的安全边界，传统面向人类用户的 OS 交互契约已经吃力。
3. 三层架构：ANOLISA 叠加在传统 Linux 之上，通过系统优化层、运行时层、封装交互层把 OS 能力转成 Agent 可用服务。



图 17: 结尾源页主要呈现产品文档和钉钉交流群二维码，因此这里使用教学综合图压缩全篇逻辑：环境探索带来成本痛点，运行时契约提出新 OS 需求，三层架构承接系统能力，少想/少装/少传、Shell 与高密运行形成关键能力，GitHub 开源、龙蜥 Skill Hub 和 ECS 全地域上线完成生态落地。

视频时间：00:19:45–00:20:12

4. Token 路径：少想减少环境探索，少装减少无关上下文，少传减少输入输出冗余，让同样任务消耗更低。
5. 统一 gateway：Copilot Shell 把语法式命令负担转成意图式调用入口，让人和 Agent 通过一致接口触达底层能力。
6. 高密度运行：企业里每名员工可能配备 10 到 100 个 Agent，系统必须处理同机并发、互不干扰、内存调度、突发算力和长稳运行。
7. 三层安全：执行前拦截、执行中监控、OS 层隔离与安全基线共同决定 Agent 能拿到多大自主执行边界。
8. 生态承载：云产品集成、轻量应用服务器、GitHub 开源和龙蜥 Skill Hub 把底座能力扩展到开发者与社区。

一条主线看 ANOLISA

ANOLISA 面向的是一组运行时系统问题：成本、接口、性能、安全和生态必须一起处理。只优化其中一项，Agent 仍会在真实企业环境里遇到落地瓶颈。

这条链也提示读者，评估 Agent 系统时不能只看模型能力。模型越强，越有可能被赋予更多工具和更长任务链；任务链越长，环境契约、执行证据和隔离边界越重要。真正的工程问题，是怎样把“会推理的模型”放进一个“可运行、可控、可审计、可扩展”的系统里。

8.5 读者检查清单：评估自己的 Agent 工作负载

如果读者正在建设自己的 Agent 应用，可以用下面的清单做一次务实评估。

1. 环境探索比例：统计 Agent 的轮次和 token，有多少花在目录探测、依赖确认、权限试错、日志搜索上。若 w_{env} 长期偏高，说明运行时契约需要补强。

2. 上下文装入量：检查每次任务装入了多少无关文档、工具说明、历史日志和配置。上下文越臃肿，成本越高，误读风险也越高。
3. 输入输出冗余：观察 Agent 与工具之间是否反复传递大段无关文本。能够结构化返回的错误、状态、资源列表，应尽量结构化返回。
4. Shell gateway 入口：确认 Agent 是否通过统一 gateway 调用 OS 能力。散落的脚本、临时命令和私有工具会让审计、权限和错误处理变得脆弱。
5. 并发密度：估算团队规模和每人 Agent 数量，评估同机并发、内存占用、长稳运行和快速定位能力。
6. 安全审计：确认每个关键动作都有 Agent 身份、任务 ID、工具名、资源对象、风险标签和处置结果。
7. OS 隔离基线：检查文件系统、网络、进程、系统调用、凭据和云资源权限是否有硬边界。模型层提醒无法替代 OS 层约束。
8. 生态治理：如果引入外部 Skill，必须记录来源、版本、权限需求、测试证据和撤回路径。

落地时最容易低估的部分

很多 Agent 原型在单机、单用户、低权限环境里看起来顺畅，一旦进入企业环境，就会撞上凭据管理、跨项目权限、合规审计、多 Agent 并发和供应链治理。评估时要按生产环境压力设计，避免按演示环境体验做判断。

8.6 本章小结

讲者的结尾把 ANOLISA 放进更大的生态图景：它已经接入阿里云产品生态，轻量应用服务器将搭载它，GitHub 开源提供可贡献入口，OpenAnolis / 龙蜥 Skill Hub 承载 Skill 与 Agent 生态，UNIX 的接口思想提供了愿景类比，Alibaba Cloud Linux 4 Agentic 版在 ECS 全地域上线则给出可用性信号。

全篇最终形成一条闭环：Agent 的真实痛点来自环境探索和运行时摩擦；ANOLISA 用运行时契约定义目标，用三层架构承载 OS 能力，用少想、少装、少传降低 token 成本，用统一 gateway 暴露系统服务，用高密 Agent 运行时支撑企业并发，用三层安全划定自主执行边界，最后通过云产品、开源社区和 Skill Hub 扩展生态。读者评估自己的 Agent 工作负载时，应同时看成本、接口、并发、安全和生态治理五个维度。